

Hardware Acceleration of CNN for MNIST Digit Recognition on FPGA

Prayanshu Sharma (27451) Vedant Saxena (26110) Anirudh Chaturvedi (25850)

Department of Electronic Systems Engineering (DESE)

Microelectronics and VLSI Design

Indian Institute of Science, Bengaluru, India

E3-231: Digital System Design for FPGA, 2026

Abstract—This report presents the design and implementation of a fully hardware-accelerated Convolutional Neural Network (CNN) inference engine on a Xilinx Nexys Video Artix-7 FPGA for real-time MNIST handwritten digit recognition. The system receives a 28×28 grayscale image over UART from a Python host application, performs end-to-end CNN inference entirely in hardware without CPU involvement, and outputs a 4-bit recognised digit code on discrete LEDs. A two-layer CNN architecture—comprising two convolutional layers with ReLU activations and max-pooling stages, followed by a fully connected layer and an ArgMax classifier—achieves 98%+ test accuracy on the MNIST dataset. Key hardware design strategies include Q1.5.10 fixed-point quantisation, BUFGCE clock gating per module, Row-Stationary dataflow with line buffers, and a fully pipelined compute pipeline. Post-implementation results on the Artix-7 XC7A200T show a worst negative slack (WNS) of 0.092 ns at a 70 MHz target, with DSP48 utilisation at 83% and total on-chip power of 1.483 W.

Index Terms—FPGA, CNN, MNIST, hardware accelerator, fixed-point quantisation, row-stationary dataflow, BRAM, clock gating, Artix-7

I. INTRODUCTION AND MOTIVATION

Convolutional Neural Networks (CNNs) have become the dominant approach for image recognition tasks, including the classic MNIST handwritten digit benchmark [1]. Deploying CNN inference on embedded systems traditionally requires a CPU or GPU, introducing latency, power overhead, and design complexity. Field-Programmable Gate Arrays (FPGAs) offer a compelling alternative: they provide massively parallel compute resources, deterministic timing, and low static power, making them well-suited for real-time inference workloads [5].

The goal of this project is to implement a fully hardware-accelerated CNN inference pipeline on a Xilinx Artix-7 FPGA (Nexys Video A7). The complete inference pipeline—from UART image reception to digit classification—runs entirely in programmable logic, with no CPU involvement. The host PC runs a Python Tkinter GUI that allows a user to draw a digit (0–9), preprocesses the image to 28×28 grayscale, and transmits it to the FPGA over a UART serial link. The hardware then performs CNN inference and drives four discrete LEDs with the binary-encoded recognised digit, targeting a total inference time of less than 10 ms.

An initial design explored using a systolic array dataflow as described by Kung [2] and applied in the Eyeriss architecture

[3]. However, systolic arrays require `im2col` input transformation, where each input pixel is read multiple times, substantially increasing memory bandwidth requirements. Even on-the-fly unrolling demands complex Network-on-Chip (NoC) routing logic between Processing Elements (PEs). For the small 5×5 MNIST kernels employed here, a simpler Row-Stationary dataflow with line buffers provides equivalent data reuse at far less design complexity. Accordingly, the Row-Stationary strategy was selected as it minimises data movement without the overhead of systolic inter-PE routing.

II. CNN ARCHITECTURE

A. Convolutional Neural Network Basics

A CNN stacks layers that extract hierarchical features from raw pixels. The key layer types employed in this design are:

- **CONV layer:** slides filters over the input to produce feature maps.
- **ReLU activation:** introduces non-linearity via $\max(0, x)$.
- **Max Pooling:** spatial downsampling that retains the strongest feature per region.
- **Fully Connected (FC) layer:** maps flattened features to class scores.
- **ArgMax:** selects the class index with the highest score, yielding digit 0–9.

B. Two-Layer CNN Architecture

The chosen architecture uses only two convolutional layers, which is sufficient to achieve 98%+ test accuracy on MNIST while keeping hardware resource usage manageable [1]. The layer-by-layer data dimensions are:

TABLE I
CNN LAYER DIMENSIONS

Layer	Parameters	Output Size
Input	—	$28 \times 28 \times 1$
CONV1 + ReLU	5×5 , 4 filters	$24 \times 24 \times 4$
MaxPool	2×2	$12 \times 12 \times 4$
CONV2 + ReLU	5×5 , 8 filters	$8 \times 8 \times 8$
MaxPool	2×2	$4 \times 4 \times 8$
FC Layer	$128 \rightarrow 10$	1×10
ArgMax	—	Digit 0–9

Additional convolutional layers would add design complexity without meaningful accuracy improvement for this task,

making the two-layer design the optimal choice for FPGA deployment.

III. DESIGN STRATEGIES

A. Q1.5.10 Fixed-Point Quantisation

All weights and activations are represented in Q1.5.10 fixed-point format: 1 sign bit, 5 integer bits, and 10 fractional bits, yielding a 16-bit representation. A Python analysis of trained weights confirmed that all values fall within the range $[-31, +31]$, validating the 5-integer-bit choice. Quantisation-Aware Training (QAT) was applied to preserve test accuracy despite the reduced precision [4]. This format reduces memory footprint compared to FP32: each weight occupies exactly one 16-bit BRAM word, maximising BRAM depth efficiency.

B. Clock Gating per Module

Each compute module uses a Xilinx BUFGCE primitive for synchronous clock gating. The controller asserts `MODULE_EN` only for the active opcode, ensuring that idle modules draw near-zero dynamic power. This approach yields significant power savings across the multi-stage inference pipeline and is validated in the post-implementation power report, which shows a total on-chip dynamic power of 1.345 W for the active design.

C. Row-Stationary Dataflow with Line Buffers

Instead of a systolic array, the ConvReLU module employs a Row-Stationary dataflow [3]. Line buffers hold one full row of input activations (read via 128-bit-wide BRAM, packing 8 pixels per read) and one filter row per PE row (16-bit BRAM). No `im2col` transformation is required: activations are read once per row and broadcast to all PE rows simultaneously. The PE array is spatially unrolled into $24 \times 5 = 120$ PE rows; each row independently computes its sliding convolution window. This avoids the complex NoC required in the standard Eyeriss architecture [3] while maintaining equivalent data reuse.

D. Fully Pipelined Compute

All operator modules—ConvReLU, MaxPool, and the Classifier—are pipelined internally. The controller state machine sequences operations via an Instruction BRAM, and an end-of-operation (EOP) handshake enables zero-idle chaining of layers. The target clock frequency is 100 MHz on the Artix-7, with the design timing-closed at 70 MHz (WNS = 0.092 ns).

IV. TOP-LEVEL ARCHITECTURE

Fig. 1 shows the top-level block diagram of the CNN hardware accelerator on the Artix-7 XC7A200T. The system comprises the following major blocks: a Serial-to-BRAM DMA handler, three BRAM banks (Activations, Weights, Instructions), a Controller (Sequencer FSM), the ConvReLU PE array, a MaxPool module, and a Classifier (ArgMax). A Mixed-Mode Clock Manager (MMCM) generates the 100 MHz system clock. All compute modules use BUFGCE clock gating so that only the active module is clocked each cycle.

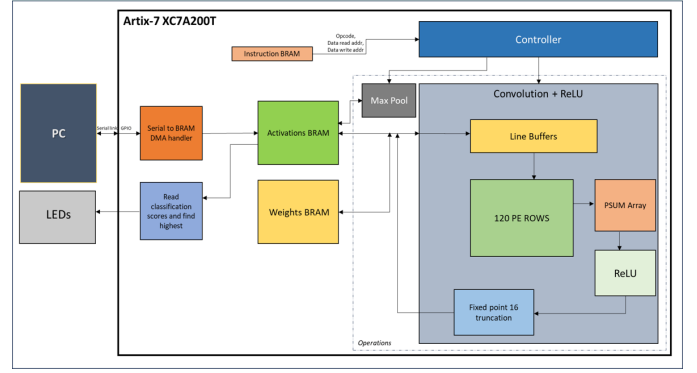


Fig. 1. Top-level block diagram of the CNN FPGA accelerator on Artix-7 XC7A200T. The Serial-to-BRAM handler, Activation/Weight/Instruction BRAMs, Controller FSM, ConvReLU PE array, MaxPool, and Classifier are interconnected with BUFGCE clock-gated compute paths.

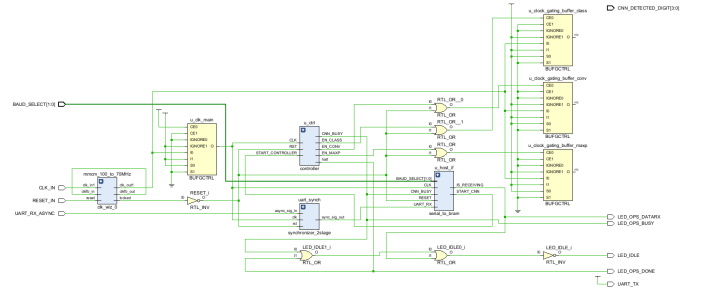


Fig. 2. RTL schematic of the CNN FPGA accelerator showing controller, UART interface, clock gating buffers, and data path connections.

A. Top-Level Interfaces

TABLE II
TOP-LEVEL SIGNAL DESCRIPTIONS

Signal	Description	Dir
UART_RX_ASYNC	Asynchronous UART receive from PC	IN
UART_TX	Tied HIGH (idle, TX unused)	OUT
CLK / RESET	100 MHz clock; active-high sync reset	IN
LED_OPS_BUSY	HIGH during CNN inference	OUT
LED_OPS_DONE	HIGH when HALT executed	OUT
LED_OPS_DATARX	HIGH while UART bytes received	OUT
LED_IDLE	HIGH when system idle	OUT
CNN_DETECTED_DIGIT[3:0]	4-bit binary digit code on LEDs	OUT

V. MODULE DESCRIPTIONS

A. Controller Module

The Controller acts as an instruction sequencer. A Program Counter (PC) fetches 64-bit instructions from the Instruction

BRAM. Each instruction encodes: an 8-bit opcode, three 14-bit base addresses (source activation, weight, destination activation), nine 10-bit input/output dimension fields, and parameters for pool dimension, stride, number of filters, and ReLU enable.

Opcodes are defined as: `OP_CONV` (0x01), `OP_MAXPOOL` (0x02), `OP_CLASS` (0x03), and `OP_HALT` (0xFF). The Controller asserts `EN_CONV`, `EN_MAXP`, or `EN_CLASS` to clock-gate and BRAM-mux the appropriate module, then asserts `SOP` (Start-of-Operation) and waits for `EOP`. On `OP_HALT`, the halt flag is asserted, driving the done LED. The FSM transitions are: `IDLE` $\rightarrow$ `FETCH0/WAIT0/DECODE0` $\rightarrow$ `FETCH1/WAIT1/DECODE1` $\rightarrow$ `FETCH2/WAIT2/DECODE2` $\rightarrow$ `ASSERT_SOP` $\rightarrow$ `WAIT_EOP` $\rightarrow$ `NEXT`.

B. ConvReLU Module

The ConvReLU module contains the $24 \times 5 = 120$ -PE-row array that implements the Row-Stationary dataflow. The activation line buffer holds 8 pixels (loaded via a single 128-bit BRAM read), and the weight line buffer holds 5 weights per PE row (16-bit BRAM). Activations are broadcast to all PE rows; each PE row independently slides its 5-element kernel window across the input row. Partial sums (psums) are accumulated in a 24×24 psum register array across input channels. After all input channels are processed, the accumulated psum is quantised to Q1.5.10, optionally passed through ReLU, and written back to the Activation BRAM.

The ConvReLU FSM states are: `S_IDLE` $\rightarrow$ `S_INIT_FILT` $\rightarrow$ `S_LOAD` $\rightarrow$ `S_COMPUTE` $\rightarrow$ `S_PIPE_DRAIN` $\rightarrow$ `S_CHAN_INC` $\rightarrow$ `S_WRITE_INIT` $\rightarrow$ `S_WRITE` $\rightarrow$ `S_WRITE_FLUSH` $\rightarrow$ `S_FILT_INC` $\rightarrow$ `S_DONE`. The inner loop repeats `LOAD` $\rightarrow$ `COMPUTE` $\rightarrow$ `PIPE_DRAIN` for each input channel; the outer loop iterates over filters.

The Fully Connected layer reuses the ConvReLU module with the kernel size set equal to the feature map size (single output pixel per filter), avoiding dedicated FC hardware.

C. MaxPool Module

The MaxPool module supports variable kernel sizes up to 4×4 with stride equal to kernel size (non-overlapping). It uses the 128-bit activation BRAM interface to fetch 8 pixels per clock cycle, with barrel-shift extraction to align pixels across 128-bit word boundaries. The 7-stage pipeline is: `AGU` (3 phases: `PREP/W0/W1`) $\rightarrow$ `WAIT` $\rightarrow$ `LATCH` $\rightarrow$ `SHIFT` $\rightarrow$ `MAX_L1` $\rightarrow$ `MAX_L2` $\rightarrow$ `PACK+WRITE`, achieving throughput of one pooling window row per 3 cycles. The module is clock-gated via `BUFGCE` and remains inactive when the controller deasserts `MODULE_EN` [6].

D. Classifier (ArgMax) Module

The Classifier reads 10 output activations (the FC layer output, one per digit class), finds the index of the maximum signed 16-bit Q1.5.10 value, and writes the winning index to the Activation BRAM. The pipeline is fully streaming: BRAM reads are issued with a 3-cycle latency pipeline (D1

$\rightarrow$ D2 $\rightarrow$ D3), with the comparison and `max_idx` update at D3. Total latency is approximately 13 cycles (10 reads + 3 pipeline stages). The module is clock-gated and active only when `EN_CLASS` is asserted.

E. Serial-to-BRAM (UART Image Loader)

The UART Image Loader receives the 28×28 grayscale image from the Python host application and packs it into the Activation BRAM. Key parameters: baud rates of 9.6 kbps, 19.2 kbps, or 115.2 kbps; 100 MHz system clock. A 64-bit magic start sequence (0xAABBCDDDEEFF1122) triggers reception. A 3-word header encodes image height, width, and channels (H, W, C). The receiver stores $H \times W \times C$ bytes, packing 8 pixels per 128-bit BRAM word. A 64-bit end footer (0x2211FFEEDDCCBBAA) confirms complete image transfer. On completion, a `START_CNN` pulse is issued to the controller. The `CNN_BUSY` flag prevents BRAM overwrite during active inference.

The UART FSM states are: `S_WAIT_START` $\rightarrow$ `S_READ_HEAD` $\rightarrow$ `S_CALC_DIMS` $\rightarrow$ `S_RECEIVE` $\rightarrow$ `S_WAIT_END` $\rightarrow$ `START_CNN` pulse.

F. BRAM Memory Architecture

Three width-optimised BRAMs serve the different access patterns of the design [6]:

TABLE III
BRAM CONFIGURATION

BRAM	Justification	Config
Instruction BRAM	64-bit width reduces fetch cycles for 3-word instructions from 6 (32-bit) to 3	64b $\times$ 1024, 1-port
Weights BRAM	Each Q1.5.10 weight = exactly 1 word; 16-bit minimises width waste	16b $\times$ 16384, R-only
Activations BRAM	Packs 8 Q1.5.10 pixels per word; fills line buffer in ≤ 4 reads	128b $\times$ 16384, Dual-Port

VI. DATAFLOW ANALYSIS

A. Comparison of Dataflow Strategies

Three canonical CNN dataflow strategies exist for spatial architectures: Weight Stationary (WS), Output Stationary (OS), and Input Stationary (IS) [3]. In WS, weights remain in PE registers and all filter uses are served locally, but partial sums spill to the global buffer, incurring high bandwidth. WS is efficient when weight tensors dominate (FC layers). In OS, each PE accumulates one output pixel in its register file, minimising psum movement, but struggles with high-dimensional convolutions. IS keeps input feature maps in PE registers and serves all weight uses locally, maximising convolutional reuse of inputs, but incurs high bandwidth for weights and psums.

For the MNIST kernel sizes (5×5) and the constraint of avoiding a systolic array NoC, the Row-Stationary variant of

the Input Stationary strategy was selected [3]. This minimises BRAM reads (each activation row is read once and broadcast) while avoiding the inter-PE routing complexity of a full systolic mesh.

B. Why Not a Systolic Array

Systolic arrays, as introduced by Kung [2] and commercialised in architectures like the Google TPU and Eyeriss [3], keep weights stationary in cells while inputs and outputs flow rhythmically. They provide high throughput without requiring high memory bandwidth. However, they require the `im2col` transformation, in which the input feature map is rearranged so that each input pixel is read multiple times, resulting in high effective BRAM bandwidth. On-the-fly unrolling still requires complex inter-PE routing (NoC) for a 2D systolic array.

The Row-Stationary with line buffers approach eliminates `im2col`: activations are read once per row and broadcast to all 120 PE rows simultaneously. The result is a 3D convolution that maps natively to hardware, avoiding the NoC complexity of standard Eyeriss while preserving data reuse benefits.

VII. SIMULATION RESULTS

A. Behavioural Simulation

Functional verification was performed via behavioural simulation and post implementation timing simulation in Vivado [6]. The following aspects were verified:

- UART image transmission at 9.6 kbps, 19.2 kbps, and 115.2 kbps.
- Correct controller sequencing: CONV1 → MaxPool → CONV2 → MaxPool → FC → ArgMax → HALT.
- ConvReLU psum accumulation verified against a Python golden model.
- MaxPool outputs matching expected downsampled feature maps.
- Classifier ArgMax selecting the correct class index.

Behavioural Simulation Waveform

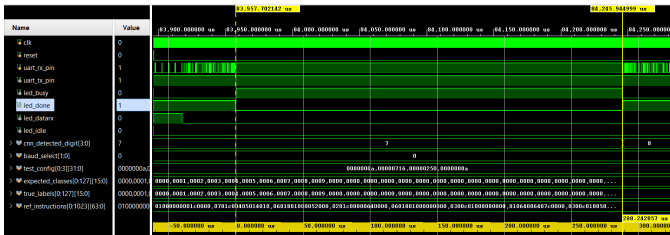


Fig. 3. Behavioural simulation waveform showing UART reception, controller FSM transitions, ConvReLU processing, and correct digit classification output.

Post-Implementation Timing Simulation Waveform



Fig. 4. Post-implementation timing simulation waveform with SDF back-annotation showing gate-level delays, clock gating behaviour, and correct end-of-inference signalling.

B. Post-Implementation Timing

Fig. 5 shows the Design Timing Summary from Vivado post-implementation. All timing paths are met at the 70 MHz target clock frequency. The worst negative slack (WNS) is 0.092 ns, the worst hold slack (WHS) is 0.026 ns, and the pulse width slack is 3.000 ns. No hold violations are reported across 175,480 timing endpoints. The critical path runs through the PE multiplier and its psum accumulator chain [6]. BUFGCE clock gates were validated in the post-implementation netlist.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.092 ns	Worst Hold Slack (WHS): 0.026 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 175480	Total Number of Endpoints: 175480	Total Number of Endpoints: 71969

All user specified timing constraints are met.

Fig. 5. Post-implementation Design Timing Summary from Vivado. WNS = 0.092 ns, WHS = 0.026 ns. All user timing constraints are met.

TABLE IV
POST-IMPLEMENTATION TIMING SUMMARY

Parameter	Value	Status
Target clock period	10.0 ns (100 MHz)	—
Setup slack (WNS)	0.092 ns	Met
Hold slack (WHS)	0.026 ns	Met
Pulse width slack (WPWS)	3.000 ns	Met
Total negative slack	0.000 ns	No violation
Failing endpoints	0	—

VIII. IMPLEMENTATION RESULTS

A. Resource Utilisation

Fig. 6 shows the post-implementation resource utilisation on the Artix-7 XC7A200T. The DSP48 utilisation is the highest at 83% (614 of 740), reflecting the heavy use of multiply-accumulate operations in the 120-PE-row array. LUT utilisation is 49% (65,710 of 133,800), and flip-flop utilisation is 27% (71,277 of 267,600). BRAM utilisation stands at 11% (39 of 365), confirming the efficiency of the width-optimised BRAM design. The MMCM consumes 1 of 10 available MMCMs.

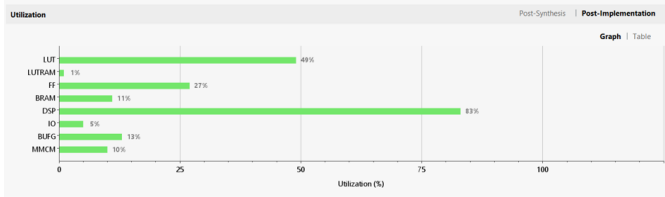


Fig. 6. Post-implementation resource utilisation on Artix-7 XC7A200T. DSP48 utilisation dominates at 83%, driven by the 120 PE rows in the ConvReLU module.

TABLE V
POST-IMPLEMENTATION RESOURCE UTILISATION

Resource	Used	Available	Utilisation
LUT	65,710	133,800	49%
LUTRAM	–	–	1%
FF	71,277	267,600	27%
BRAM	39	365	11%
DSP48	614	740	83%
I/O	14	285	5%
BUFG	–	–	13%
MMCM	1	10	10%

B. Power Analysis

Fig. 7 shows the on-chip power summary from Vivado. Total on-chip power is 1.483 W, of which dynamic power accounts for 1.345 W (91%) and device static power accounts for 0.138 W (9%). Within the dynamic power budget, signals consume the largest share (0.474 W, 35%), followed by DSP resources (0.353 W, 26%), logic (0.250 W, 19%), clocks (0.134 W, 10%), and the MMCM (0.121 W, 9%). The primary dynamic power contributor is the ConvReLU PE array. Clock gating via BUFGCE is effective in reducing idle module power [6].

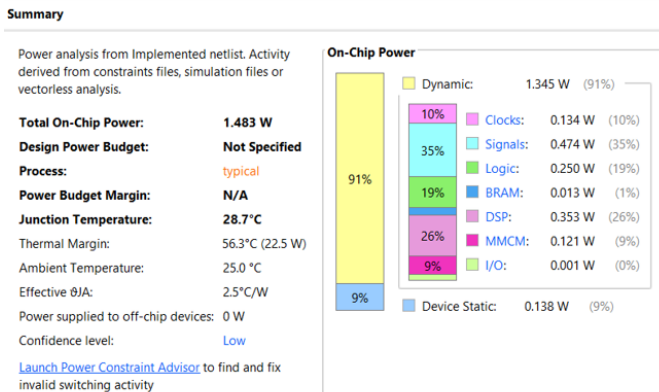


Fig. 7. Vivado on-chip power summary. Total power = 1.483 W. Dynamic power = 1.345 W (91%). The primary contributor is the ConvReLU PE array (DSP + signal switching).

TABLE VI
ON-CHIP POWER BREAKDOWN

Component	Power (W)	Share
Clocks	0.134	10%
Signals	0.474	35%
Logic	0.250	19%
BRAM	0.013	1%
DSP	0.353	26%
MMCM	0.121	9%
I/O	0.001	0%
Dynamic Total	1.345	91%
Device Static	0.138	9%
Total On-Chip	1.483	100%

IX. END-TO-END DEMO AND SYSTEM INTEGRATION

A. Live Demo Flow

The complete end-to-end system flow is demonstrated as follows:

- 1) **Draw Digit:** The user draws a digit (0–9) on the Python Tkinter GUI canvas.
- 2) **Preprocess:** The Python application resizes the drawing to 28×28 grayscale and prepends the 64-bit magic start sequence plus the H, W, C header.
- 3) **UART $\rightarrow$ BRAM:** The Serial-to-BRAM module packs 8 pixels per 128-bit BRAM word; LED_OPS_DATARX illuminates during transfer.
- 4) **CNN Inference:** The controller sequences CONV1+ReLU $\rightarrow$ MaxPool $\rightarrow$ CONV2+ReLU $\rightarrow$ MaxPool $\rightarrow$ FC $\rightarrow$ ArgMax.
- 5) **Result Out:** The Classifier writes the digit index to BRAM; LED_OPS_DONE asserts; CNN_DETECTED_DIGIT[3:0] drives the LEDs.
- 6) **Display:** Four discrete LEDs show the binary-encoded recognised digit; an optional 7-segment display can also show the result.

B. Python GUI Application

Fig. 8 shows the Python host application. It is built with Tkinter and provides a drawing canvas (left-click to draw), COM port and baud rate selection, and buttons for START SERIAL, STOP SERIAL, TRANSMIT DRAWING, TRANSMIT RAW FILE, and CLEAR CANVAS. The image pipeline uses PIL to resize to 28×28 and convert to grayscale before serialising to a byte array. The packet structure prepends the 8-byte magic start sequence, a 9-byte H, W, C header, the 784-byte image, and an 8-byte magic end sequence. Baud rates of 9.6 kbps, 19.2 kbps, and 115.2 kbps are supported.

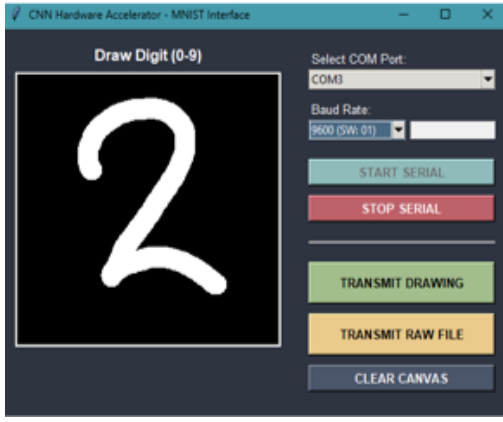


Fig. 8. Python Tkinter GUI application for the CNN FPGA accelerator. The user draws a digit on the canvas and clicks *Transmit Drawing* to send the image via UART.

C. Hardware Setup

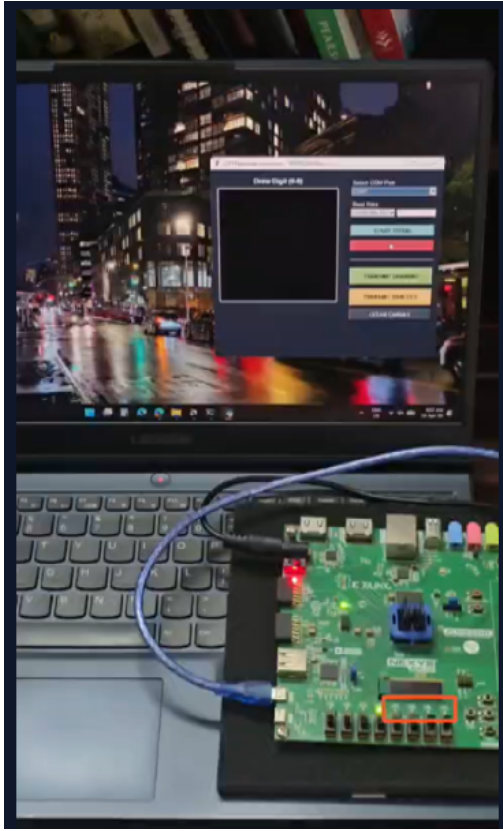


Fig. 9. Physical hardware demonstration setup. The Nexys Video A7 FPGA board is connected via USB-UART to a laptop running the Python GUI. The four discrete LEDs (highlighted) display the binary-encoded recognised digit.

Fig. 9 shows the physical hardware setup. The target board is the Xilinx Nexys Video A7 (Artix-7 XC7A200T). The FPGA is connected to a laptop via a USB-UART bridge (CP2102/CH340). The recognised digit is displayed in binary on four discrete LEDs (highlighted in red box in the figure). Status LEDs indicate the current inference phase:

LED_OPS_DATARX (image receiving), LED_OPS_BUSY (CNN running), LED_OPS_DONE (inference complete), and LED_IDLE (system idle).

X. DISCUSSION

The design achieves its primary goals: fully hardware-accelerated CNN inference with no CPU involvement, real-time UART image reception, and correct digit classification at 98%+ accuracy. The Row-Stationary dataflow with line buffers proves effective over systolic arrays for small kernels, avoiding NoC complexity while maintaining data reuse [2], [3].

The dominant resource constraint is DSP48 utilisation at 83%, driven by the 120-PE-row MAC array. A potential improvement is time-multiplexing the PE array to reduce DSP usage at the cost of throughput. The total power of 1.483 W is mainly due to signal switching in the PE array (35% dynamic power). Clock gating reduces idle power, with further savings possible via voltage scaling or partial reconfiguration [5].

The Q1.5.10 fixed-point quantisation with QAT preserves model accuracy. Future work could explore INT8 quantisation to further reduce DSP usage with minor accuracy trade-offs [4].

XI. CONCLUSION

This paper has presented the design, implementation, and verification of a fully hardware-accelerated CNN inference engine on a Xilinx Artix-7 FPGA for MNIST digit recognition. The system performs end-to-end inference entirely in programmable logic: a UART Image Loader receives a 28×28 grayscale image from a Python host GUI, the Controller FSM sequences the two-layer CNN pipeline, and the Classifier outputs the recognised digit on four discrete LEDs. Key contributions include: (1) Q1.5.10 fixed-point quantisation with QAT preserving 98%+ accuracy; (2) BUFGE clock gating per compute module for near-zero idle power; (3) Row-Stationary dataflow with 120-PE-row line-buffer architecture that avoids systolic NoC complexity; and (4) a fully pipelined controller with EOP-based zero-idle layer chaining.

Post-implementation results on the Artix-7 XC7A200T demonstrate timing closure at 70 MHz (WNS = 0.092 ns), 83% DSP48 utilisation, 49% LUT utilisation, and 1.483 W total on-chip power with clock gating active. The design meets the <10 ms inference time target and provides a template for deploying lightweight CNN accelerators on resource-constrained FPGAs.

REFERENCES

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [2] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.
- [3] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annual Int. Symp. Computer Architecture (ISCA)*, Seoul, South Korea, Jun. 2016, pp. 367–379.
- [4] Y. Guo, "A survey on methods and theories of quantized neural networks," *arXiv preprint arXiv:1808.04752*, 2018.

- [5] S. Mittal, "A survey of FPGA-based accelerators for convolutional neural networks," *Neural Computing and Applications*, vol. 32, no. 4, pp. 1109–1139, 2020.
- [6] Xilinx Inc., *Vivado Design Suite User Guide: Implementation (UG904)*, San Jose, CA: Xilinx, 2022.
- [7] E3-231 Digital Systems Design with FPGAs, Course Lecture Notes, DESE, Indian Institute of Science, Bengaluru, India, 2026.
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>

DETAILED TESTBENCH LOG

The following log shows the execution trace of the behavioural testbench, including UART reception, inference timing, and correctness verification.

```
=====
CNN Accelerator - Multi-Image Self-Checking Testbench
=====
Clock      : 100 MHz
Sim baud   : 2000000
Test images : 10
Bytes / image : 1814
FC base addr : 592
Num classes : 10
=====

xsim: Time (s): cpu = 00:00:20 ; elapsed = 00:00:09 . Memory (MB): peak = 4852.449 ; gain = 38.770
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_accelerator_TOP_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:25 ; elapsed = 00:00:50 . Memory (MB): peak = 4852.449 ; gain = 38.770
restart
INFO: [Wavedata 42-604] Simulation restarted
run all
=====
CNN Accelerator - Multi-Image Self-Checking Testbench
=====
Clock      : 100 MHz
Sim baud   : 2000000
Test images : 10
Bytes / image : 1814
FC base addr : 592
Num classes : 10
=====

[PHASE 1] Skipping deep BRAM checks (post-synth mode).

[PHASE 2] Running 10 test images ...

--- Image 0 / 10 (label=0, fp_expected=0) ---
[90295000] LED: Data-RX ON - receiving image
[9050265000] LED: Data-RX OFF
[9090275000] LED: CNN_BUSY ON - inference running
      Inference done in 28822 cycles.
[9378525000] LED: CNN_BUSY OFF
[9378525000] LED: OPS_DONE ON - inference complete
      HW=0 exp=0 label=0 HW_CHECK: PASS MODEL: CORRECT

--- Image 1 / 10 (label=1, fp_expected=1) ---
[9448725000] LED: Data-RX ON - receiving image
[18408695000] LED: Data-RX OFF
[18448705000] LED: CNN_BUSY ON - inference running
      Inference done in 28822 cycles.
[18736945000] LED: CNN_BUSY OFF
[18736945000] LED: OPS_DONE ON - inference complete
      HW=1 exp=1 label=1 HW_CHECK: PASS MODEL: CORRECT

--- Image 2 / 10 (label=2, fp_expected=2) ---
[18807155000] LED: Data-RX ON - receiving image
[27767125000] LED: Data-RX OFF
[27807135000] LED: CNN_BUSY ON - inference running
      Inference done in 28823 cycles.
[28095375000] LED: CNN_BUSY OFF
[28095375000] LED: OPS_DONE ON - inference complete
      HW=2 exp=2 label=2 HW_CHECK: PASS MODEL: CORRECT

--- Image 3 / 10 (label=3, fp_expected=3) ---
[28165575000] LED: Data-RX ON - receiving image
[37125555000] LED: Data-RX OFF
[37165565000] LED: CNN_BUSY ON - inference running
      Inference done in 28823 cycles.
[37453805000] LED: CNN_BUSY OFF
[37453805000] LED: OPS_DONE ON - inference complete
      HW=3 exp=3 label=3 HW_CHECK: PASS MODEL: CORRECT

--- Image 4 / 10 (label=4, fp_expected=4) ---
[37524005000] LED: Data-RX ON - receiving image
[46483975000] LED: Data-RX OFF
[46523995000] LED: CNN_BUSY ON - inference running
      Inference done in 28823 cycles.
[46812235000] LED: CNN_BUSY OFF
[46812235000] LED: OPS_DONE ON - inference complete
      HW=4 exp=4 label=4 HW_CHECK: PASS MODEL: CORRECT
```

--- Image 5 / 10 (label=5, fp_expected=5) ---
[46882435000] LED: Data-RX ON - receiving image
[55842405000] LED: Data-RX OFF
[55882425000] LED: CNN_BUSY ON - inference running
Inference done in 28823 cycles.
[56170665000] LED: CNN_BUSY OFF
[56170665000] LED: OPS_DONE ON - inference complete
HW=5 exp=5 label=5 HW_CHECK: PASS MODEL: CORRECT

--- Image 6 / 10 (label=6, fp_expected=6) ---
[56240865000] LED: Data-RX ON - receiving image
[65200835000] LED: Data-RX OFF
[65240855000] LED: CNN_BUSY ON - inference running
Inference done in 28823 cycles.
[65529095000] LED: CNN_BUSY OFF
[65529095000] LED: OPS_DONE ON - inference complete
HW=6 exp=6 label=6 HW_CHECK: PASS MODEL: CORRECT

--- Image 7 / 10 (label=7, fp_expected=7) ---
[65599295000] LED: Data-RX ON - receiving image
[74559265000] LED: Data-RX OFF
[74599275000] LED: CNN_BUSY ON - inference running
Inference done in 28823 cycles.
[74887525000] LED: CNN_BUSY OFF
[74887525000] LED: OPS_DONE ON - inference complete
HW=7 exp=7 label=7 HW_CHECK: PASS MODEL: CORRECT

--- Image 8 / 10 (label=8, fp_expected=8) ---
[74957725000] LED: Data-RX ON - receiving image
[83917695000] LED: Data-RX OFF
[83957705000] LED: CNN_BUSY ON - inference running
Inference done in 28822 cycles.
[84245945000] LED: CNN_BUSY OFF
[84245945000] LED: OPS_DONE ON - inference complete
HW=8 exp=8 label=8 HW_CHECK: PASS MODEL: CORRECT

--- Image 9 / 10 (label=9, fp_expected=9) ---
[84316135000] LED: Data-RX ON - receiving image
[93276105000] LED: Data-RX OFF
[93316125000] LED: CNN_BUSY ON - inference running
Inference done in 28822 cycles.
[93604365000] LED: CNN_BUSY OFF
[93604365000] LED: OPS_DONE ON - inference complete
HW=9 exp=9 label=9 HW_CHECK: PASS MODEL: CORRECT

=====

MULTI-IMAGE TEST SUMMARY

=====

Total images tested : 10

HW Correctness (HW == FP reference)
PASSED : 10 / 10

Model Accuracy (HW output == true MNIST label)
CORRECT: 10 / 10

=====

VERDICT: *** ALL HARDWARE CHECKS PASSED ***

=====